

ORCA – The platform primitives: patterns and use cases

What gets built in `platform/`, what each thing is called in the literature, and the concrete case each one serves.

Written for engineers who have not used this project shape before. Every pattern here has a name and a body of writing behind it — the names are given so you can read the source material rather than take this document's word for it.

1 • What this project setup is called

LAYER	THE PATTERN	WHERE TO READ ABOUT IT
The repository	Monorepo, Gradle multi-project – many independently deployable services, one repository, one version catalog	Gradle's own multi-project documentation
<code>platform/</code>	Internal Spring Boot starters – libraries with auto-configuration that services depend on and that configure themselves	Spring Boot reference: <i>Creating Your Own Auto-configuration</i>
Inside <code>orca-runtime</code>	Modular monolith – modules with enforced boundaries inside one deployable	<i>Spring Modulith</i> documents this shape well, even though we enforce it with ArchUnit rather than adopting the library
<code>api / domain / persistence</code>	Layered architecture , with the dependency rule pointing inward	Any DDD text; also <i>ports and adapters</i> if you want the stricter form
<code>build-checks/</code>	Architectural fitness functions – automated tests that fail the build when the architecture is violated	<i>Building Evolutionary Architectures</i> (Ford, Parsons, Kua); the tool is ArchUnit
<code>contracts/ per service</code>	API-first (also <i>design-first</i>) – the specification is authored, and the code is generated from it	OpenAPI Generator, <code>spring</code> generator with <code>interfaceOnly</code>

The one to look up first is **fitness functions**. It is the least familiar idea and it is what makes this architecture hold: the rules are not documented and reviewed, they are executed and they fail the build.

2 • The five primitives, by name

MODULE	THE PATTERN IT IMPLEMENTS	ALSO KNOWN AS
<code>outbox</code>	Transactional Outbox + Polling Publisher	Chris Richardson's microservices patterns; the claim uses <code>SELECT ... FOR UPDATE SKIP LOCKED</code>
<code>lease</code>	Lease with a fencing token	Martin Kleppmann, <i>Designing Data-Intensive Applications</i> – the fencing token section is the canonical treatment
<code>scope</code>	Ambient scope + a repository guard	Closest named forms: a Hibernate filter, or the Specification pattern applied centrally
<code>idempotency</code>	Idempotent Receiver / Idempotency Key	Stripe's idempotency-key documentation is the clearest public write-up
<code>web</code>	Response envelope + ambient principal	RFC 7807 is a near relative for the error half

3 • What actually gets built, and the case it serves

platform/outbox

Built:

- An `outbox` table — sequence, ordering key, event type, payload, timestamp — plus per-consumer acknowledgement rows.
- `OutboxWriter.write(orderingKey, eventType, payload)` — called **inside the same transaction** as the business write.
- `OutboxRelay` — a poller that claims rows with `SELECT ... FOR UPDATE SKIP LOCKED`, delivers to each registered consumer, and records the acknowledgement.
- A `ConsumerRegistry` naming who must acknowledge before a row is deletable.

The case it serves. `orca-runtime` finishes a visit. In one transaction it writes the execution row **and** an outbox row saying `visit.completed`. The relay picks it up and delivers to `orca-portal`, which closes the ticket. If the relay is down for an hour, the row waits. If the process dies between the two writes, neither exists.

Why a service cannot do this itself: the moment the notification is a separate call after the commit, the two can disagree — and nothing detects it.

platform/lease

Built:

- A `service_lease` table keyed `(service, lease_name)` — holder, acquired, renewed, expires, **fence token**.
- `LeaseManager.acquire(name, holderId, duration)` → an optional lease carrying its token. Acquisition is a **conditional update**, never read-then-write.
- `renew` and `release`.
- A guarded write helper: a write that presents its fence token and is **rejected** if the token is stale.

The case it serves. `orca-edge` runs on two instances. Lane 3's camera talks to one address, so exactly one instance may own that lane's connection. Instance A holds the lease. Its network stalls; the lease expires; instance B acquires it with a **higher token**. Instance A recovers and tries to finish writing a capture — its token is old, and the write is rejected.

Why the fence token matters more than the lease: you cannot guarantee a stalled process is dead. You can guarantee its writes are refused.

platform/scope

Built:

- `ScopeContext` — the site or customer the current request is acting for, established at the request boundary.
- A repository base or query seam that applies the scope predicate to every query.
- **An ArchUnit rule that fails the build** on any query constructed outside the seam.
- Default deny: an unset scope returns **zero rows**, never all rows.

The case it serves. A clerk opens the work-item queue. `workItemRepository.findQueued()` returns their site's items. Nobody wrote `where site_id = ?`, and nobody could have forgotten it — a query written outside the seam does not compile.

Why it is a primitive: the current system has roughly **816 hand-written scope conditions**. One omission is a cross-tenant leak with no error and no log line. Correctness that depends on everyone remembering does not survive a decade.

platform/idempotency

Built:

- An idempotency table — key, operation, recorded outcome, timestamp.
- `IdempotencyStore.begin(key)` returning **new**, **in progress**, or **completed with the recorded result**.
- `complete(key, outcome)`.

The case it serves. `orca-runtime` sends a gate-open command carrying a command id. The barrier rises, but the response is lost. Runtime retries. The store returns the **recorded outcome** — so the barrier rises once, and the retry gets the answer it was missing rather than an error.

Why "return the outcome" rather than "reject duplicates": the caller retried *because it never saw the first answer*.

`409 Duplicate` is the one response that cannot help it.

platform/web

Built:

- `ApiResponse<T>` — one envelope: status, code, message, data, errors, pagination metadata, request id.
- `ApiError` and a machine-readable `ErrorCode`.
- A global exception handler that maps everything to the envelope and **never serialises an exception or a schema name**.
- `SystemContext` — an explicit identity for entry points no user invoked.

The case it serves. A partner's integration calls the event API and gets a validation failure. It branches on `code`, not on the wording of `message`, so improving the message never breaks their integration. Meanwhile the outbox relay — which no user invoked — runs under an explicit system identity, so its writes can be authorised and attributed rather than being anonymous.

4 • How a service uses them

A service depends on the primitives it needs and gets configuration for free:

```
dependencies {
    implementation(project(":platform:outbox"))
    implementation(project(":platform:scope"))
    implementation(project(":platform:web"))
}
```

Each primitive is an **auto-configuration** — a service adds the dependency and the beans are wired, in the same way `spring-boot-starter-data-jpa` configures a `DataSource` without being told.

In use, they are almost invisible:

- A service writes its business row and calls `outboxWriter.write(...)` in the same transaction.
- A repository extends the scoped base and its queries are scoped.
- A controller returns `ApiResponse.of(...)`.
- A handler that needs exclusive work asks `leaseManager.acquire(...)` and checks whether it got it.

That invisibility is the point. **The correct thing is the easy thing, and the incorrect thing does not compile.**

5 • What is deliberately not in platform/

No entity, service or helper that names a **visit**, a **lane**, a **ticket**, a **driver** or a **truck**. No shared DTOs between services. No "common utilities" module.

A **build check enforces this**, and it is the check most worth keeping strict. A shared module that accumulates domain logic becomes the thing every service depends on and nobody can change — which is a well-travelled way for a monorepo to become a distributed monolith.
